LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 4 & 5

OPP & ENCAPSULATION

Disusun Oleh:

M Rizki Ruswan 105224018

PROGRAM STUDI ILMU KOMPUTER

FAKULTAS SAINS DAN KOMPUTER

UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Pada tugas ini saya membuat sebuah program simulasi sistem reservasi kamar hotel yang bertujuan untuk membantu pengelolaan pemesanan kamar secara lebih terstruktur dan aman. Sistem reservasi hotel perlu dirancang dengan baik karena sering terjadi kesalahan operasional, seperti jumlah tamu yang melebihi kapasitas kamar, kesalahan penentuan harga, maupun penggunaan voucher yang tidak sesuai dengan ketentuan yang berlaku. Oleh karena itu, sistem harus mampu menangani berbagai kemungkinan kesalahan input agar data yang tersimpan tetap valid dan sesuai aturan.

Dalam program ini saya menggunakan bahasa pemrograman Java dengan menerapkan konsep Object-Oriented Programming (OOP) yang telah dipelajari, khususnya prinsip Encapsulation. Konsep ini digunakan agar data penting pada objek kamar tidak dapat diakses secara langsung dari luar kelas, melainkan harus melalui metode tertentu yang telah dilengkapi validasi. Dengan cara ini, sistem dapat mencegah perubahan data yang tidak sesuai dan menjaga konsistensi informasi kamar.

Program yang saya buat juga dilengkapi dengan beberapa fitur penting, seperti validasi tipe kamar, pembatasan harga minimum, pengecekan kapasitas tamu sebelum pemesanan dilakukan, serta perhitungan total pembayaran dengan dukungan kode voucher promo. Selain itu, metode overloading digunakan untuk memberikan variasi proses pemesanan dan perhitungan pembayaran agar program menjadi lebih fleksibel. Melalui program ini diharapkan simulasi sistem reservasi kamar hotel dapat berjalan dengan baik serta mampu meminimalkan kesalahan operasional yang mungkin terjadi.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	nomorKamar	String	Menyimpan nomor identitas kamar hotel
2	tipeKamar	String	Menyimpan jenis kamar (Reguler, Premium, Suite)
3	kapasitasMaksimal	int	Menentukan jumlah maksimum tamu dalam kamar
4	hargaPerMalam	double	Menyimpan harga sewa kamar per malam
5	isTersedia	boolean	Menunjukkan status ketersediaan kamar

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	KamarHotel()	Constructor	Membuat objek kamar dengan data pendaftaran kilat
2	KamarHotel(..., harga)	Constructor	Membuat objek kamar dengan data lengkap
3	getNomorKamar()	Functional	Mengambil nomor kamar
4	getTipeKamar()	Functional	Mengambil tipe kamar
5	getKapasitasMaksimal()	Functional	Mengambil kapasitas maksimal
6	getHargaPerMalam()	Functional	Mengambil harga kamar
7	getIsTersedia()	Functional	Mengambil status ketersediaan kamar
8	setTipeKamar()	Procedural	Mengatur tipe kamar dengan validasi input
9	setHargaPerMalam()	Procedural	Mengatur harga dengan batas minimal
10	pesanKamar()	Procedural	Memesan kamar tanpa parameter
11	pesanKamar(int)	Procedural	Memesan kamar dengan validasi jumlah tamu
12	batalPesan()	Procedural	Membatalkan pemesanan kamar
13	hitungTotalBayar(int)	Functional	Menghitung total biaya tanpa voucher
14	hitungTotalBayar(int,String)	Functional	Menghitung total biaya dengan voucher
15	cetakInfo()	Procedural	Menampilkan informasi lengkap kamar

IV. Dokumentasi dan Pembahasan Code

1.

```java
public class KamarHotel {

    private String nomorKamar;
    private String tipeKamar;
    private int kapasitasMaksimal;
    private double hargaPerMalam;
    private boolean isTersedia;
```

Penjelasan:

Di sini saya membuat kelas KamarHotel sebagai fondasi utama untuk mengelola data kamar dalam sistem kita. Selanjutnya, saya mendefinisikan atribut seperti nomorKamar dan tipeKamar menggunakan tipe data String, serta kapasitasMaksimal dengan int. Saya juga menyertakan hargaPerMalam dalam format double untuk presisi biaya dan isTersedia sebagai boolean untuk memantau status ketersediaan kamar secara *real-time*.

2.

```
public KamarHotel(String nomorKamar, String tipeKamar, int kapasitasMaksimal) {
    this.nomorKamar = nomorKamar;
    setTipeKamar(tipeKamar);
    this.kapasitasMaksimal = kapasitasMaksimal;
    this.hargaPerMalam = 0;
    this.isTersedia = true;
}
```

Penjelasan:

Selanjutnya, saya membuat sebuah constructor untuk kelas ini agar kita bisa langsung menginisialisasi objek kamar dengan data yang diperlukan. Di sini, saya memetakan parameter nomorKamar dan kapasitasMaksimal ke atribut kelas menggunakan kata kunci this, sementara untuk tipeKamar, saya memanggil method setTipeKamar agar validasinya tetap terjaga. Saya juga menetapkan nilai awal hargaPerMalam sebesar 0 dan mengatur status isTersedia menjadi true, yang menandakan bahwa setiap kamar baru yang kita buat akan otomatis berstatus siap digunakan.

3.

```
    public KamarHotel(String nomorKamar, String tipeKamar,
                      int kapasitasMaksimal, double hargaPerMalam) {

        this.nomorKamar = nomorKamar;
        setTipeKamar(tipeKamar);
        this.kapasitasMaksimal = kapasitasMaksimal;
        setHargaPerMalam(hargaPerMalam);
        this.isTersedia = true;
    }
```

Penjelasan:

Selanjutnya, saya membuat variasi *constructor* kedua yang lebih lengkap agar kita bisa langsung menentukan harga saat objek dibuat. Dalam blok ini, saya memproses parameter nomorKamar dan kapasitasMaksimal secara langsung, namun untuk tipeKamar dan hargaPerMalam, saya sengaja memanggil fungsi setTipeKamar serta setHargaPerMalam. Langkah ini saya ambil untuk memastikan bahwa data harga

yang dimasukkan sudah melalui prosedur validasi yang tepat sebelum disimpan, sementara status isTersedia tetap saya atur secara otomatis ke nilai true.

4.

```java
    public String getNomorKamar() {
        return nomorKamar;
    }

    public String getTipeKamar() {
        return tipeKamar;
    }

    public int getKapasitasMaksimal() {
        return kapasitasMaksimal;
    }

    public double getHargaPerMalam() {
        return hargaPerMalam;
    }

    public boolean getIsTersedia() {
        return isTersedia;
    }
```

Penjelasan:

Selanjutnya, saya mengimplementasikan serangkaian getter methods untuk setiap atribut yang telah didefinisikan sebelumnya. Karena atribut-atribut tersebut bersifat private, kita memerlukan metode publik seperti getNomorKamar, getTipeKamar, hingga getIsTersedia agar bagian lain dari program dapat mengakses nilainya tanpa bisa mengubahnya secara langsung. Saya memastikan setiap fungsi mengembalikan tipe data yang sesuai—baik itu String, int, double, maupun boolean—sehingga integritas data di dalam sistem kita tetap terjaga dengan baik.

5.

```java
    public void setTipeKamar(String tipeKamar) {

        if (tipeKamar.equalsIgnoreCase(anotherString: "Reguler") ||
            tipeKamar.equalsIgnoreCase(anotherString: "Premium") ||
            tipeKamar.equalsIgnoreCase(anotherString: "Suite")) {

            this.tipeKamar = tipeKamar;
        } else {
            System.out.println(x: "Tipe kamar tidak valid, Diubah menjadi Reguler.");
            this.tipeKamar = "Reguler";
        }
    }
```

Penjelasan:

Selanjutnya, saya mengimplementasikan method setTipeKamar yang berfungsi sebagai validasi data agar sistem kita tidak menerima input sembarangan. Di dalam kode ini, saya menggunakan struktur pengondisian if untuk memeriksa apakah input yang diberikan sesuai dengan kategori yang diizinkan, yaitu "Reguler", "Premium", atau "Suite". Dengan bantuan fungsi equalsIgnoreCase, kita memastikan input tetap valid tanpa mempedulikan penggunaan huruf kapital. Jika input tidak sesuai dengan salah satu kategori tersebut, maka saya menyetel nilai default ke "Reguler" dan menampilkan pesan peringatan agar pengguna tahu bahwa data tersebut telah otomatis disesuaikan oleh sistem.

6.

```java
    public void setHargaPerMalam(double hargaPerMalam) {

        if (hargaPerMalam < 50000) {
            System.out.println(x: "Harga terlalu rendah, Diubah menjadi 50000.");
            this.hargaPerMalam = 50000;
        } else {
            this.hargaPerMalam = hargaPerMalam;
        }
    }
```

Penjelasan:

Selanjutnya, saya mengimplementasikan method setTipeKamar yang berfungsi sebagai validasi data agar sistem kita tidak menerima input sembarangan. Di dalam kode ini, saya menggunakan struktur pengondisian if untuk memeriksa apakah input yang diberikan sesuai dengan kategori yang diizinkan, yaitu "Reguler", "Premium", atau "Suite". Dengan bantuan fungsi equalsIgnoreCase, kita memastikan input tetap valid tanpa mempedulikan penggunaan huruf kapital. Jika input tidak sesuai dengan salah satu kategori tersebut, maka saya menyetel nilai default ke "Reguler" dan menampilkan pesan peringatan agar pengguna tahu bahwa data tersebut telah otomatis disesuaikan oleh sistem.

7.

```java
    public void pesanKamar() {
        if (isTersedia) {
            isTersedia = false;
            System.out.println(x: "Kamar berhasil dipesan.");
        } else {
            System.out.println(x: "Kamar sudah terisi!");
        }
    }
```

Penjelasan:

Selanjutnya, saya mengimplementasikan logika untuk proses pemesanan melalui method pesanKamar. Di sini, saya menggunakan struktur if-else untuk

mengecek status ketersediaan kamar; jika variabel isTersedia bernilai true, maka saya akan mengubah statusnya menjadi false dan memberikan konfirmasi bahwa kamar berhasil dipesan. Namun, jika ternyata kamar sudah tidak tersedia, saya akan menampilkan pesan peringatan bahwa kamar tersebut sudah terisi, sehingga sistem kita dapat mencegah terjadinya pemesanan ganda pada satu kamar yang sama.

8.

```java
    public void pesanKamar(int jumlahTamu) {

        if (!isTersedia) {
            System.out.println(x: "Kamar sudah dipesan");
            return;
        }

        if (jumlahTamu > kapasitasMaksimal) {
            System.out.println(x: "Jumlah tamu melebihi kapasitas, Pesanan ditolak.");
        } else {
            isTersedia = false;
            System.out.println("Pemesanan berhasil untuk "
                    + jumlahTamu + " tamu.");
        }
    }

```

Penjelasan:

Selanjutnya, saya mengembangkan versi overload dari method pesanKamar yang lebih spesifik dengan menambahkan parameter jumlahTamu. Dalam logika ini, saya melakukan dua tahap pengecekan: pertama, saya memastikan apakah kamar tersebut masih tersedia menggunakan kondisi !isTersedia; jika sudah terisi, maka proses akan langsung dihentikan dengan perintah return. Selanjutnya, saya menambahkan validasi kapasitas untuk memastikan bahwa jumlahTamu yang datang tidak melebihi kapasitasMaksimal yang dimiliki kamar tersebut. Jika semua syarat terpenuhi, maka saya akan mengubah status ketersediaan menjadi false dan menampilkan pesan keberhasilan pemesanan, sehingga sistem kita menjadi lebih aman dan terkontrol.

9.

```java
    public void batalPesan() {
        isTersedia = true;
        System.out.println(x: "Pesanan dibatalkan, kamar tersedia kembali.");
    }

    public double hitungTotalBayar(int jumlahMalam) {
        return jumlahMalam * hargaPerMalam;
    }

    public double hitungTotalBayar(int jumlahMalam, String kodeVoucher) {

        double total = jumlahMalam * hargaPerMalam;

        if (kodeVoucher.equals(anObject: "PROMO") && jumlahMalam >= 3) {
            System.out.println(x: "Voucher berhasil, Diskon 20% diberikan.");
            return total * 0.8;
        } else {
            System.out.println(x: "Voucher gagal digunakan.");
            return total;
        }
    }
```

Penjelasan:

Selanjutnya, saya melengkapi kelas ini dengan fitur pembatalan dan penghitungan biaya. Melalui method batalPesan, saya mengembalikan status isTersedia menjadi true agar kamar bisa dipesan kembali oleh tamu lain. Selanjutnya, saya membuat dua versi overloading untuk hitungTotalBayar. Versi pertama hanya mengalikan jumlahMalam dengan hargaPerMalam untuk tagihan standar.

Namun, pada versi kedua, saya menambahkan parameter kodeVoucher untuk memberikan fleksibilitas pada sistem kita. Di sini, saya menyisipkan logika pengecekan: jika tamu menggunakan kode "PROMO" dan menginap minimal 3 malam, maka saya akan memberikan diskon sebesar 20% (dikali 0.8). Jika syarat tersebut tidak terpenuhi, saya akan mengembalikan nilai total normal sehingga pendapatan hotel kita tetap akurat dan terjaga.

10.

```
    public void cetakInfo() {
        System.out.println("===== STRUK INFORMASI KAMAR =====");
        System.out.println("Nomor Kamar      : " + nomorKamar);
        System.out.println("Tipe Kamar       : " + tipeKamar);
        System.out.println("Kapasitas Maks   : " + kapasitasMaksimal);
        System.out.println("Harga / Malam    : " + hargaPerMalam);
        System.out.println("Status Tersedia  : " + isTersedia);
        System.out.println("----------------------------------");
    }
}
```

Penjelasan:

Selanjutnya, sebagai bagian penutup dari kelas ini, saya membuat *method* cetakInfo untuk menyajikan seluruh data kamar dalam format yang rapi dan mudah dibaca. Di dalam metode ini, saya menggunakan perintah System.out.println untuk menampilkan identitas kamar secara detail, mulai dari nomorKamar, tipeKamar, kapasitasMaksimal, hingga hargaPerMalam. Saya juga menyertakan status ketersediaan agar kita bisa melihat dengan cepat apakah kamar tersebut siap dihuni atau tidak. Dengan adanya struk informasi ini, kita telah berhasil membangun sistem manajemen kamar hotel yang lengkap dan terorganisir.

11.

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {

        KamarHotel kamar1 =
                new KamarHotel(nomorKamar: "101", tipeKamar: "Reguler", kapasitasMaksimal: 3);

        kamar1.setTipeKamar(tipeKamar: "Presidential");
        kamar1.setHargaPerMalam(-10000);

        KamarHotel kamar2 =
                new KamarHotel(nomorKamar: "202", tipeKamar: "Premium", kapasitasMaksimal: 2, harg… 300000);

        kamar2.pesanKamar(jumlahTamu: 4);

        kamar2.pesanKamar(jumlahTamu: 2);

        kamar2.pesanKamar();

        System.out.println(x: "\nTagihan kamar1:");
        double total1 = kamar1.hitungTotalBayar(jumlahMalam: 2, kodeVoucher: "PROMO");
        System.out.println("Total bayar: " + total1);

        System.out.println(x: "\nTagihan kamar2:");
        double total2 = kamar2.hitungTotalBayar(jumlahMalam: 4, kodeVoucher: "PROMO");
        System.out.println("Total bayar: " + total2);

        kamar1.cetakInfo();
        kamar2.cetakInfo();
    }
}
```

Penjelasan:

Selanjutnya, sebagai tahap penyelesaian, saya membuat kelas Main untuk menguji seluruh fitur yang telah kita bangun. Di dalam metode utama ini, saya menginstansiasi dua objek kamar, yaitu kamar1 dan kamar2, dengan detail yang berbeda. Saya mencoba melakukan berbagai skenario, seperti mengubah tipe kamar ke "Presidential" dan harga ke nilai negatif untuk melihat bagaimana sistem kita menangani validasi, serta melakukan simulasi pemesanan dengan jumlah tamu yang melebihi kapasitas pada kamar2.

Selanjutnya, saya menghitung total tagihan menggunakan sistem voucher "PROMO" untuk melihat apakah diskon 20% berhasil diterapkan sesuai durasi menginap. Sebagai langkah terakhir, saya memanggil method cetakInfo() untuk kedua kamar tersebut guna menampilkan laporan akhir di layar, sehingga kita bisa memastikan bahwa seluruh logika program, mulai dari inisialisasi hingga perhitungan biaya, telah berjalan dengan sempurna.

V. Kesimpulan

Secara keseluruhan, saya telah berhasil mengimplementasikan prinsip Pemrograman Berorientasi Objek (OOP) dalam membangun sistem manajemen KamarHotel. Melalui praktikum ini, kita dapat melihat bagaimana konsep *encapsulation* diterapkan untuk melindungi data melalui akses *private* dan penggunaan *getter-setter*. Selanjutnya, saya juga menerapkan *method overloading* pada fungsi pemesanan dan penghitungan biaya, yang memberikan fleksibilitas bagi sistem kita dalam menangani berbagai skenario tamu, termasuk penerapan validasi kapasitas dan sistem diskon otomatis menggunakan kode voucher. Sebagai penutup, pengujian pada

kelas Main membuktikan bahwa logika program yang saya ketik telah berjalan dengan stabil, mulai dari pengaturan awal hingga pelaporan status kamar secara akurat.

IV. Daftar Pustaka

Oracle. (2023). *Java Documentation*.https://docs.oracle.com/en/java/